

Introduction

Based on the functional and non-functional requirements identified in LO1, this artifact documents the testing plan, which serves as a starting point for Test-Driven Development. Given the nature of the project, requirements and specifications will evolve to adapt to the project's needs.

Unlike the Waterfall methodology, Agile software development does not require all requirements to be defined prior to development. Instead, it follows a continuous cycle of development and testing. Coupled with Test-Driven Development, Agile is a well-suited approach for this project.

Basic Structure of the System

MedSupplyDrones will be built using the Spring Boot framework, which organises the codebase into a Controller layer, a Service layer, and optionally a Persistence layer (Database). Since the system needs to fetch live data to assign the most appropriate drone to fulfil multiple orders simultaneously, a Persistence layer won't be necessary.

The Controller Layer

The Controller layer acts as the interface between the ILP REST Service and the Medication Management System, which contains the list of orders. Logic in this layer should be kept to a minimum. Testing will focus on verifying that endpoints return the correct output and HTTP status codes.

The Service Layer

The Service layer defines the application's boundary and the operations available to interfacing client layers. It encapsulates business logic, controls transactions, and coordinates responses. Because most of the application logic resides here, this layer will require extensive testing including unit tests, integration tests, and regression tests to validate any additions or changes in requirements.

Given the structure of the system and the functional and non-functional requirements, the testing strategy will follow this approach:

1. **Unit Testing** - Create unit tests before implementation so that each piece of functionality is validated before being integrated into the codebase.
2. **Integration Testing** - Once unit tests pass, integration testing begins to ensure that each layer interacts correctly with the others (Medication Management System > ILP REST Service > Controller Layer > DTO > Service Layer).
3. **System Testing** - After unit and integration testing are complete, system-level testing ensures that the system meets all requirements from a user's perspective.

If any tests fail, they should be addressed immediately. The same applies to any additions or changes made to the specification.

Functional Requirements

1. Testing environment configuration:

1. Check that the service reads the ILP_URL environment variable correctly on startup.
2. Check that a missing ILP_URL environment variable produces an appropriate error message.

3. Check that a malformed URL in ILP_URL is detected and produces an error.
= 3 tests

2. Testing fetching data from ILP REST Service:

1. Check that when the ILP REST server is unreachable, an appropriate error is returned.
2. Check that /actuator/health/livenessState returning non-"UP" status is handled correctly.
3. Check that invalid/malformed JSON from /drones endpoint triggers appropriate error handling.
4. Check that invalid/malformed JSON from /service-points endpoint triggers appropriate error handling.
5. Check that invalid/malformed JSON from /restricted-areas endpoint triggers appropriate error handling.
6. Check that invalid/malformed JSON from /drones-for-service-points endpoint triggers appropriate error handling.
7. Check data is fetched fresh before each calculation (not cached from previous requests).

= 7 tests

3. Testing geometry calculations:

1. Check Pythagorean distance calculation between two coordinates is correct.
2. Check closeness test returns **true** when distance is *strictly less than* 0.00015°.
3. Check closeness test returns **false** when distance is *exactly* 0.00015° (boundary case).
4. Check closeness test returns **false** when distance is greater than 0.00015°.
5. Check point-in-polygon correctly identifies a point **inside** a no-fly zone.
6. Check point-in-polygon correctly identifies a point **outside** a no-fly zone.
7. Check point-in-polygon correctly handles a point **on the boundary** of a no-fly zone.
8. Check line-segment intersection correctly detects a path crossing a no-fly zone boundary.
9. Check line-segment intersection correctly identifies when a path does **not** cross a boundary.
10. Check that crossing a **corner** of a no-fly zone is correctly detected as invalid.

= 10 tests

4. Testing drone filtering and matching:

1. Check filtering by **cooling capability** returns only drones with cooling: true when order requires cooling.
2. Check filtering by **heating capability** returns only drones with heating: true when order requires heating.
3. Check filtering by **capacity** returns only drones with sufficient capacity for order size.
4. Check filtering by **maxMoves** returns only drones that can complete the required flight path.
5. Check filtering by **availability (dayOfWeek)** returns only drones available on the specified day.
6. Check filtering by **availability (from/until time)** returns only drones available within the time window.

7. Check that when no drone matches all requirements, an empty list is returned.
8. Check that drones are correctly associated with their service points.

= 8 tests

5. Testing flight path calculation:

1. Check all adjacent moves in any calculated path are exactly 0.00015° apart (within $\pm 10^{-12}$ tolerance).
2. Check the path starts at the assigned service point location.
3. Check the path ends at the same service point it started from.
4. Check the path passes "close" to (within 0.00015° of) each pick-up point.
5. Check the path passes "close" to (within 0.00015° of) each drop-off point.
6. Check drone hovers at each pick-up point (two consecutive moves at same coordinate).
7. Check drone hovers at each drop-off point (two consecutive moves at same coordinate).
8. Check **no move coordinate is inside** a no-fly zone.
9. Check **no move segment crosses** a no-fly zone boundary.
10. Check the total number of moves does not exceed the drone's maxMoves limit.
11. Check path correctly routes around no-fly zones when direct path is blocked.

= 11 tests × N paths

6. Testing cost calculation:

1. Check total cost formula: $costInitial + costFinal + (moves \times costPerMove)$ is correctly applied.
2. Check cost calculation uses the correct drone-specific constants.
3. Check cost is correctly summed when multiple drones are used.

= 3 tests

7. Testing order assignment and optimisation:

1. Check multiple orders to nearby drop-off points are assigned to the same drone flight.
2. Check orders with incompatible requirements (e.g., cooling vs heating) are assigned to different drones.
3. Check total order size does not exceed assigned drone's capacity.
4. Check the system attempts to minimise total moves across all flights.
5. Check the system attempts to minimise total cost across all flights.

= 5 tests

8. Testing REST API endpoints:

1. Check valid order submission returns HTTP 200 with computed route in JSON format.
2. Check malformed JSON request returns HTTP 400 Bad Request.
3. Check request to non-existent endpoint returns HTTP 404 Not Found.
4. Check /actuator/health endpoint returns "UP" when service is healthy.
5. Check response JSON schema matches expected format (paths, costs, statistics).

6. Check appropriate error response when order cannot be fulfilled (no available drones).

= 6 tests

Non-Functional Requirements

1. **Performance** – execution time shall not exceed 30 seconds for any task. Time the execution and assert.
2. **Determinism** – identical requests produce identical outputs. Run same request multiple times and compare.
3. **Docker deployment** – service runs correctly when deployed as Docker container. Test with exported tar image.

= 3 tests

Total theoretical number of high-level tests = 3 + 7 + 10 + 8 + 11 + 3 + 5 + 6 + 3 = 56 tests (plus N × path-specific tests)

This plan provides a comprehensive list of tests covering all components at different levels with reasonable coverage. The total number of tests is likely to increase to accommodate for different techniques such as combinatorial testing and boundary value analysis.

Issues

External Service Dependency: The ILP REST service is outside our control. Its behaviour and responses cannot be guaranteed to remain consistent, potentially impeding test reliability. Mitigation: Use mock server for testing.

Floating-Point Precision: Cumulative rounding errors over many moves could cause position drift. The specification allows $\pm 10^{-12}$ tolerance, which must be verified in long path calculations.

Dynamic Data: Server data can change between calls. Test cases must account for the fact that repeated calls may return different drone availability, requiring fresh data fetch before each calculation.

Instrumentation

To aid the testing process, helper methods, assertions, and scaffolding in the form of stubs (mock-ups of parts of the system) will be developed. Diagnostic output will be utilised to explore subsystem behaviour.

Instrumentation to be created:

1. **Reusable flight path validator** that checks attributes for any path provided as argument:
 1. Checks all adjacent moves are exactly 0.00015° apart.
 2. Checks no path coordinate is inside any no-fly zone (takes zone list as argument).
 3. Checks no path segment crosses any no-fly zone boundary.
 4. Checks path visits all required points (pick-ups and drop-offs) within tolerance.
2. **Timing instrumentation** to measure execution time of path calculations and assert < 30 seconds.
3. **Closeness assertion helper** to verify two coordinates are within 0.00015° of each other.

4. **Cost calculation verifier** that independently computes expected cost from path and drone parameters.

Scaffolding:

1. **Mock ILP REST Server** (using WireMock) for internal testing:
 - Returns configurable drone data with various capabilities
 - Returns configurable service point locations and availability schedules
 - Returns hardcoded no-fly zones of various shapes for boundary testing
 - Can simulate error conditions (timeouts, malformed responses, 500 errors)
2. **Test Data Builders** for generating test fixtures:
 - DroneBuilder – creates Drone objects with configurable capabilities
 - OrderBuilder – generates order JSON with configurable locations and requirements
 - ZoneBuilder – creates no-fly zone polygons for various test scenarios
3. **Docker Test Container** using Testcontainers to spin up the application in Docker for integration testing.

Evaluation

The mock ILP REST server scaffolding will be the most time-consuming task and should be designed for reusability. The test data builders provide flexibility in creating edge-case scenarios.

Possible downsides: Hardcoded test examples might not be representative of all possible user inputs. However, they will address immediate edge cases and can be extended as new scenarios are discovered.

Strengths: The instrumentation enables isolated testing without external dependencies, deterministic test execution, and reusable validation logic that simplifies writing new tests.

Areas for improvement: Consider adding mutation testing (PIT) to verify test quality, contract testing (Pact) to verify API expectations, and visual path rendering for debugging complex routes.

The described instrumentation will provide reusable tools and simplify as well as strengthen the testing process, allowing the development team to maintain high confidence in the system's correctness as requirements evolve.